OpenTelemetry Web 101

Instrumentation of web apps

Apr 2026

About me

David Luna
Principal Software Engineer - APM Team - Elastic

// Contact Information

Email: david.luna@elastic.co
GitHub: github.com/david-luna

// Biography

Passionate about OpenTelemetry, observability, and building scalable web applications. Formerly developer of web and mobile applications.

elastic

Real User Monitoring (RUM) Players

Dynatrace	Datadog	New Relic	Splunk	Elastic
AppDynamics	Raygun	SpeedCurve	Catchpoint	IBM Instana

elastic

What is OpenTelemetry?

Observability framework designed to generate, export and collect telemetry.

100% open-source and vendor agnostic.

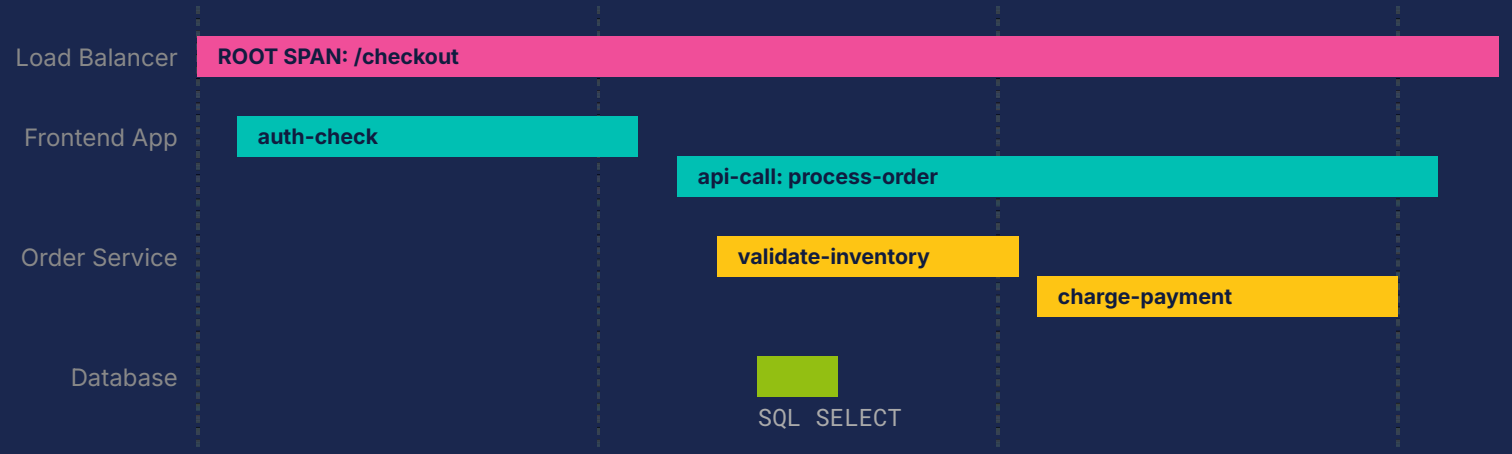
Not an observability backend. You own the data and decide where it goes.

elastic

Tracing: Following the Journey of a Single Request

Traces provide a complete, end-to-end picture of how requests move through distributed systems, breaking down latency into individual spans.

```
// Standardized vendor-agnostic OTLP protocol
span.setAttribute("customer_id", "c-123");
```



elastic

OpenTelemetry Signals: Metrics

Metrics are numerical measurements of service health and performance over time. Unlike traces, they provide aggregated visibility into system state and resource utilization.

Key Metric Types:

- **Counters:** Value that only increases (e.g., total requests, jobs processed).
- **Gauges:** Current instantaneous value (e.g., queue depth, memory usage).
- **Histograms:** Statistical distribution of values (e.g., latency percentiles).

```
// OTLP Gauge Example
meter.gaugeBuilder("queue.depth")
```

elastic

OpenTelemetry Signals: Logs

Logs are discrete events or records that occur within a system. Unlike metrics or traces, logs provide high-cardinality, granular detail about specific moments in time.

Log Attributes:

- **Timestamp:** When the event occurred.
- **Severity:** Importance level (INFO, WARN, ERROR).
- **Body:** The actual message or structured data.

```
// OTLP Log Record Example
logger.emit({
  body: "User login successful",
  severityNumber: SeverityNumber.INFO,
})
```

elastic

OpenTelemetry Components

elastic

Instrumenting Your App

The vanilla way (only traces)

tracing.js

```
import { defaultResource, resourceFromAttributes } from '@opentelemetry/resources';
import { BatchSpanProcessor, ConsoleSpanExporter } from '@opentelemetry/sdk-trace-base';
import { OTLPTraceExporter } from '@opentelemetry/exporter-trace-otlp-http';
import { WebTracerProvider } from '@opentelemetry/sdk-trace-web';
import { ZoneContextManager } from '@opentelemetry/context-zone';

// 1. Identify the service and set metadata (e.g.: `service.name`)
const resource = defaultResource().merge(resourceFromAttributes({
  'service.name': 'my-webapp'
}));

// 2. Initialize the Tracer Provider with the defined resource
const provider = new WebTracerProvider({
  resource,
  // 3. Configure a Span processor and exporter for data egress
  spanProcessors: [new BatchSpanProcessor(new OTLPTraceExporter({
    url: 'https://collector.example.com/v1/traces'
  }))]
});

// 4. Register the Provider (optionally with a specific context manager)
provider.register({ contextManager: new ZoneContextManager()});
```

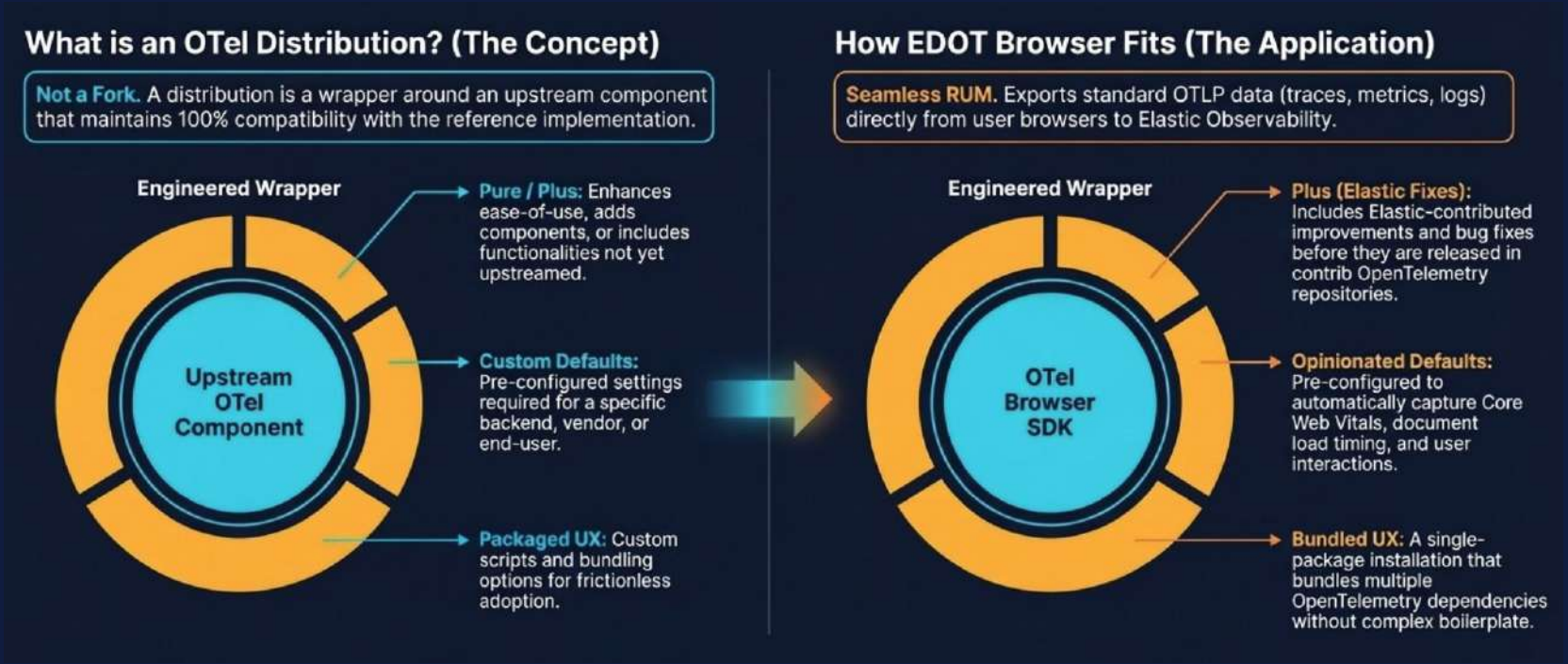
Package Management

Needs tooling

Several APIs to learn

elastic

How to make it easier?



elastic

Instrumenting Your App

EDOT Browser

tracing.js

```
import { startBrowserSdk } from '@elastic/opentelemetry-browser';

// Start passing the configuration options
startBrowserSdk({
  serviceName: 'my-webapp',
  otlpEndpoint: 'https://collector.example.com',
}));
```

Single Package

Tooling optional

Simplified API

elastic

DEMO

OpenTelemetry Browser SIG: Phase 1 Summary

Core Principles

Developing an OpenTelemetry implementation highly optimized and tailored for browser environments. Prioritizing browser-specific instrumentations.

Key Objectives

Establish a dedicated browser repository. Optimize package size and dependencies for mobile and resource-constrained environments.

Initial Instrumentation Focus

Capture vital performance metrics: Browser Navigation, Web Exceptions, User Actions, Navigation Timing, and Core Web Vitals.

Browser Fundamentals

Reviewing Tracing, Metrics, and Logs APIs for browser-specific data modeling needs like Sessions, Resources, and Navigation.

elastic

Useful links

Core Repository

The primary SDK for Node.js and Browser tracing, metrics, and logs.

`github.com/open-telemetry/opentelemetry-js`

Contrib Repository

Community-maintained instrumentations and resource detectors.

`github.com/open-telemetry/opentelemetry-js-contrib`

Browser Repository

Instrumentations and other components focused on browser-specific optimizations.

`github.com/open-telemetry/opentelemetry-browser`

Website & Docs

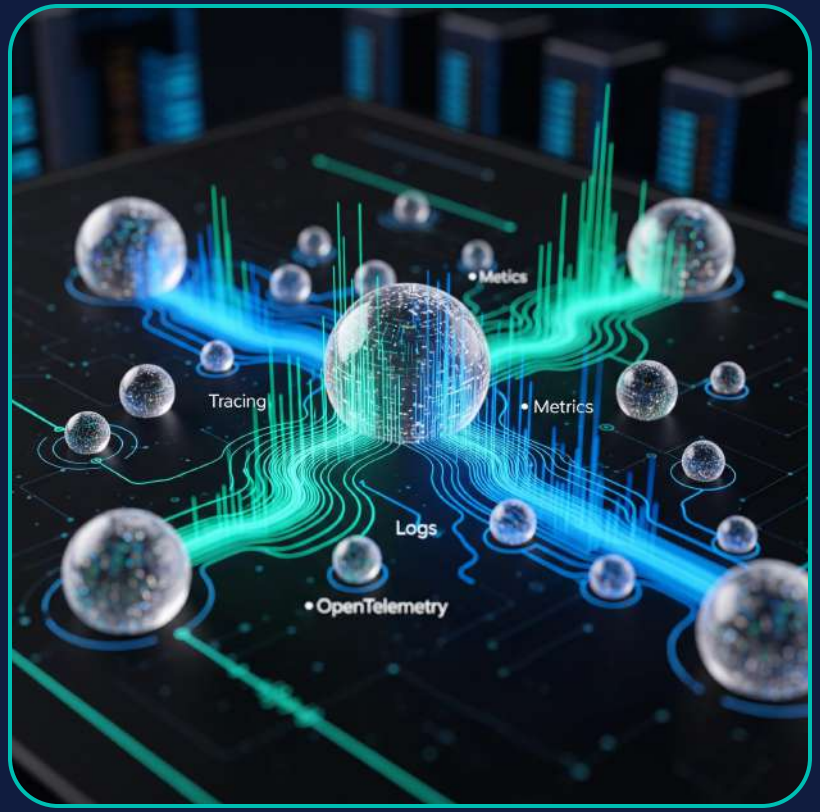
Official documentation and getting started guides for JS.

`opentelemetry.io/docs/languages/js/`

elastic

Thank You!

Questions & Discussion



elastic